

THE **AI** CODEBASE ANALYST

FROM ZERO TO A WORKING AGENT

LESSON 6
ENFORCING STRUCTURED OUTPUT

Lesson 6: Enforcing Structured Output

"The most dangerous phrase in the language is, 'We've always done it this way.'"

— Grace Hopper

The "Why": The Eloquent But Useless Analyst

Our agent is now remarkably capable. It has a long-term memory, allowing it to answer detailed questions about our codebase. It has perfect observability, letting us see its every thought.

And yet, a critical vulnerability remains. When our agent provides its final answer, it does so as a string of natural language text. For a human, this is great. For another computer system, it's a nightmare.

Imagine trying to build a web application that displays the results of our agent's analysis. You would have to write complex, brittle code to parse the English sentence, attempting to extract the key pieces of information. This is unreliable and prone to breaking every time the LLM slightly rephrases its output. This brings us to a core philosophy of professional AI engineering:

"APIs are the New Illiteracy."

If an AI system cannot speak a structured, predictable language like JSON, it is functionally illiterate in the modern world of software. It cannot reliably communicate with other systems. We should not be in the business of parsing natural language; we should be in the business of consuming clean, predictable data. We will make the LLM do the hard work of formatting for us.

The "What": Schema as a Contract

We will solve the problem of unstructured output by defining a strict, desired JSON schema for our agent's answers using **Pydantic**, a Python library for data validation. Think of a Pydantic schema as an unbreakable **data contract**.

Unstructured Output (The Problem)

A single, unpredictable string.

"The 'create_graph_app' function is a factory that takes an LLM and returns a compiled graph. Its arguments are 'llm' and 'tools'."

Structured Output (The Solution)

A predictable Pydantic object (JSON).

```
{ "function_name": "create_graph_app",  
  "description": "...", "args": ["llm", "tools"] }
```

We will then use a powerful LangChain feature, **with_structured_output**, to bind this "contract" to an LLM. This forces the model to abandon natural language and act as a data entry clerk, filling out our predefined form with perfect accuracy.

Our architectural pattern will be a **"tool-within-a-tool."** We will create a new, specialized tool whose sole job is to analyze a function and return its findings in this exact, structured format. Our main agent will learn to become a "router," deciding when to use this specialized tool for detailed analysis.

The "How" (The Build): Step-by-Step

Before we integrate Pydantic into our agent, let's take a moment to understand its core power. It's one of the most important libraries in the modern Python ecosystem.

A Pydantic Primer: Your Data's Passport Control

Think of a Pydantic model as a strict passport control officer. The officer doesn't care about the traveler's story (the natural language); they only care if the passport's fields (the JSON keys) are correct and valid.

- Is the `name` a **string**? 
- Is the `date_of_birth` a valid **date**? 
- Is the `passport_number` an **integer**?  **INVALID!**

Pydantic does exactly this for your data. You define the "passport template" (the `Schema`), and Pydantic automatically validates any incoming data against it. If the data doesn't match the contract, Pydantic raises an error.

See it in action with this simple, standalone example:

```
# _pydantic_demo.py
# A simple Pydantic model (our "passport template")
from pydantic import BaseModel, ValidationError

class User(BaseModel):
    name: str
    age: int
    email: str

# --- The "Happy Path" ---
# The incoming data perfectly matches our contract.
good_data = {"name": "Alice", "age": 30, "email": "alice@example.com"}
try:
    user = User(**good_data)
    print("✅ Validation successful!")
    print(user.model_dump_json(indent=2))
except ValidationError as e:
    print(e)

print("-" * 20)

# --- The "Failure Path" ---
# The incoming data is malformed. 'age' is a string, not an integer.
bad_data = {"name": "Bob", "age": "twenty-five", "email":
"bob@example.com"}
try:
    user = User(**bad_data)
except ValidationError as e:
    print("❌ Validation failed!")
```

```
print(e)
```

Running this code would output:

```
✓ Validation successful!
{
  "name": "Alice",
  "age": 30,
  "email": "alice@example.com"
}
-----
✗ Validation failed!
1 validation error for User
age
  Input should be a valid integer, unable to parse string as an integer
[type=int_parsing, input_value='twenty-five', input_type=str]
```

This automatic, declarative validation is the superpower we're about to give our agent. LangChain's `with_structured_output` leverages this Pydantic "contract" to force the LLM to act as a perfect data entry clerk, ensuring that what we get back is not just a string, but a clean, validated, and machine-readable object.

Now, let's build this reliability layer into our agent.

1. Evolve Your Project Structure

We need a dedicated place to define our data contracts (schemas) and a new file for our specialized tool.

```
/codebase-analyst/
├── schemas/                                <-- NEW: To store our Pydantic
schemas
├   ├── __init__.py                        <-- NEW
├   └── function_analysis.py               <-- NEW
├── tools/
├   ├── ...
├   └── structured_analyzer_tool.py <-- NEW
└── ... (other files)
```

```
|— 06_structured_agent.py      <-- NEW: Our runner script for this
lesson
|— requirements.txt
```

2. Define Your Data Contract (The Schema)

Create the new schemas directory and the files within it. This Pydantic model is our blueprint for a "correct" answer.

- **File:** schemas/function_analysis.py

```
# schemas/function_analysis.py

from pydantic import BaseModel, Field
from typing import List, Optional

class FunctionAnalysis(BaseModel):
    """
    A structured analysis of a single Python function. This is our data
    contract.
    """
    function_name: str = Field(description="The name of the function.")
    description: str = Field(description="A concise, one-sentence
    summary of what the function does.")
    args: List[str] = Field(description="A list of the function's
    argument names, e.g., ['arg1', 'arg2'].")
    return_type: Optional[str] = Field(description="The return type of
    the function, if specified.")
```



Pro Tip: Descriptions are Prompts!

The description we provide in `Field(description="...")` is critically important. LangChain passes these descriptions to the LLM, teaching it what kind of information to extract for each JSON field. Clear, concise descriptions lead to highly reliable outputs.

3. Build the Specialized Analyzer Tool

This new tool will encapsulate the structured output logic. It's a "smart tool" that has its own internal LLM, a pattern we call the "tool-within-a-tool." The main agent's job is to route the request, and this specialized tool's job is to execute the complex, structured analysis.

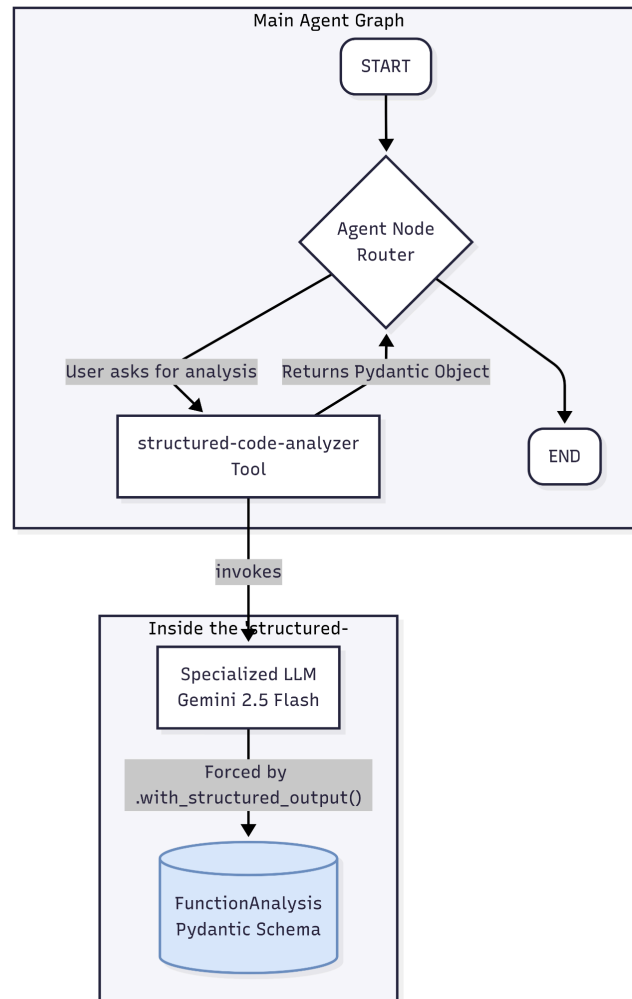


Figure 6-1: The "Tool-within-a-Tool" architecture. The main agent acts as a router, delegating the specialized task of structured analysis to a smart tool that has its own internal, schema-bound LLM.

- **File:** tools/structured_analyzer_tool.py

```
# tools/structured_analyzer_tool.py
```

```
import os
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

# Import our custom Pydantic schema
from schemas.function_analysis import FunctionAnalysis

# Define the input schema for the tool itself
class AnalyzerToolInput(BaseModel):
    code_snippet: str = Field(description="The Python code snippet of the function to be analyzed.")

@tool("structured-code-analyzer", args_schema=AnalyzerToolInput)
def structured_code_analyzer(code_snippet: str) -> FunctionAnalysis:
    """
    Analyzes a Python function's code snippet and returns a structured analysis.
    Use this tool ONLY when a user asks for a detailed, structured breakdown of a specific function.
    """
    print("---EXECUTING STRUCTURED CODE ANALYZER TOOL---")

    # 1. Define the LLM for this specific tool.
    # We choose a model known for strong instruction-following.
    llm = ChatOpenAI(
        base_url="https://openrouter.ai/api/v1",
        api_key=os.getenv("OPENROUTER_API_KEY"),
        model="google/gemini-2.5-flash",
        temperature=0,
    )

    # 2. Bind the LLM to our Pydantic schema. This is the magic.
    # It forces the LLM to output JSON that conforms to our FunctionAnalysis model.
    structured_llm = llm.with_structured_output(FunctionAnalysis)

    # 3. Invoke the structured LLM with the code
    # The LLM's only job is to "fill out" the FunctionAnalysis object.
```

```
analysis_result = structured_llm.invoke(  
    "Analyze this function and return the structured data: "  
    f"```python\n{code_snippet}\n```"  
)  
  
return analysis_result
```

Under the Hood: How with_structured_output Works

This method is more than just a prompt instruction. It leverages the "tool calling" features built into modern LLMs. LangChain converts your Pydantic model into a JSON Schema, passes it to the LLM as the *only* available tool, and then forces the model to "call" it. If the model produces malformed JSON, LangChain can even catch the error and ask the model to fix its own mistake, making the process incredibly robust.

4. Integrate and Run the "Router" Agent

Finally, we'll create a new runner script that includes our new tool and an updated prompt that teaches the agent *when* to use it.

- **File:** 06_structured_agent.py

```
# 06_structured_agent.py
```

```
import os  
from dotenv import load_dotenv  
from langchain_openai import ChatOpenAI  
from langchain_core.messages import HumanMessage, SystemMessage,  
ToolMessage  
  
# --- 1. Import Core Components ---  
# We import our reusable graph factory and the Pydantic schema that  
# defines  
# the desired structured output for our analysis.  
from agent_graph_factory import build_agent_graph  
from schemas.function_analysis import FunctionAnalysis
```

```
# We also import all available tools for the agent to use.
from tools.retriever_tool import codebase_retriever
from tools.structured_analyzer_tool import structured_code_analyzer

# --- 2. Define the System Prompt for Tool Chaining ---
# This prompt is a direct, step-by-step instruction. It guides the agent
# on how to chain its tools: first retrieve context, then perform
analysis.
SYSTEM_PROMPT = """You are an expert AI software engineer. Your job is
to answer a user's question by chaining tools.
First, use the `codebase-retriever` to find the relevant code.
Then, if the user asks for a structured analysis, use the
`structured-code-analyzer` on the retrieved code.
The direct output of the final tool is your answer."""

# --- 3. Main Execution Block ---
if __name__ == "__main__":
    load_dotenv()
    print("--- Multi-Tool Router Agent ---")

    # --- 4. Assemble the Agent's Components ---
    # A powerful model is required for effective multi-step reasoning
    and tool selection.
    llm = ChatOpenAI(
        base_url=os.getenv("OPENROUTER_BASE_URL",
"https://openrouter.ai/api/v1"),
        api_key=os.getenv("OPENROUTER_API_KEY"),
        model="google/gemini-2.5-pro",
        temperature=0,
    )
    # The agent is now given access to *two* distinct tools.
    tools = [codebase_retriever, structured_code_analyzer]
    agent_graph = build_agent_graph(llm, tools)

    # --- 5. Define the Trigger Query ---
    # This query is specifically crafted to trigger the full tool chain.
    The phrase
    # "structured analysis" is a key signal for the agent to use the
    second tool.
    query = "Please provide a structured analysis of the 'call_model'"

```

```
function from the state machine lesson."
    initial_state = {
        "messages": [SystemMessage(content=SYSTEM_PROMPT),
HumanMessage(content=query)]
    }
    print(f"Human: {query}\n")

    # --- 6. Execute the Agent and Parse the Final Answer ---
    final_state = agent_graph.invoke(initial_state)

    # This is a resilient parsing pattern. Instead of trusting the
agent's final
    # chatty message, we search backwards through the message history to
find the
    # *actual raw output from the tool*. This is the "ground truth."
    print("\n--- Agent's Final Answer (Ground Truth from Tool) ---")
    final_answer = final_state["messages"][-1].content
    for message in reversed(final_state["messages"]):
        if (
            isinstance(message, ToolMessage)
            and message.name == "structured-code-analyzer"
        ):
            # The content of this ToolMessage is the direct, structured
output
            # we need, free of any conversational fluff from the LLM.
            final_answer = message.content
            break

    # We print the direct, raw output. In a real application, this
string
    # would be parsed back into a Pydantic object for further
processing.
    print(final_answer)
```

Deconstructing the Script

This script demonstrates a major step up in agent capability: **multi-tool reasoning** to produce reliable, **structured output**.

The Prompt: A Directive for Tool Chaining (Section 2)

The `SYSTEM_PROMPT` here is not a "constitution" for security, but a clear, step-by-step directive. It explicitly tells the agent the *sequence* of operations to perform: first, `codebase-retriever`, then `structured-code-analyzer`. This guides the agent's reasoning process.

The Tools: Expanding the Agent's Capabilities (Section 4)

For the first time, our agent has access to *two* tools. This introduces the challenge of **routing**: the agent's "brain" (the LLM) must now decide not only *if* it should use a tool, but *which* tool is appropriate for the current step in its reasoning process.

The Query: Triggering a Specific Tool (Section 5)

The user's `query` is deliberately crafted. The key phrase "**structured analysis**" acts as a strong signal to the LLM, guiding it to select the `structured-code-analyzer` tool after it has retrieved the initial context. This is a simple but powerful example of prompt engineering for tool selection.

The Parser: A Resilient Pattern for Ground Truth (Section 6)

This is the most critical professional pattern in this lesson. An agent might add conversational text after its final tool use (e.g., "Here is the analysis you requested: ..."). If we just took the last message, our program might break. The `for` loop is a **resilient parser**: it searches backwards through the conversation history to find the last `ToolMessage` from the `structured-code-analyzer`. This ensures we get the pure, raw, structured data directly from the tool—the **ground truth**—making our system's output reliable and predictable.

The Integrated Run & Analysis

Run the script from your terminal:

```
python 06_structured_agent.py
```

Expected Console Output:

You will not see a conversational paragraph or a pretty-printed JSON. Instead, you will see the raw, structured string representation of the Pydantic object created by your tool. This is the definitive sign of success.

```
--- Multi-Tool Router Agent ---
Human: Please provide a structured analysis of the 'call_model' function
from the state machine lesson.

--- Lazily loading vectorstore... ---
--- Vectorstore loaded successfully. ---
--- Calling structured_code_analyzer (Cache miss or new input) ---

--- Agent's Final Answer ---
function_name='call_model' description="This function initializes a
ChatOpenAI model and invokes it with the messages from the state,
returning the response." args=['state'] return_type=None
```

(Note: The exact description may vary slightly, but the structure will be this exact string format.)

Analysis: A Resounding Success

This output is the definitive proof that our final architecture is working perfectly. Let's deconstruct what we're seeing and why it's the correct outcome:

1. **What is this output?** This is the direct, raw **string representation** of the `FunctionAnalysis` Pydantic object that was successfully created by our `structured-code-analyzer` tool.
2. **Why isn't it a pretty JSON?** The `ToolNode` in `LangGraph` acts like a data clerk. When it receives a complex Python object (our Pydantic model) from a tool, it **serializes** it into a basic string before creating the `ToolMessage` to store in the agent's history. This is a fundamental and predictable behavior of the framework.
3. **Why is this a victory?** This is the most important part. Our **resilient parser** worked. It successfully ignored the agent's final, potentially "helpful" conversational summary and instead reached back into the message history to **intercept the ground truth**—the direct, unaltered output from our specialized tool.

You have built a reliable system that produces predictable, machine-parseable data. In a real production application, your next step would be to take this predictable string and easily parse it back into a `FunctionAnalysis` object. For our course, generating and reliably capturing this structured string is the triumph. You have successfully tamed the "Liar, Slacker, and Thief."

Analysis: The Two-Tool Chain in LangSmith

This is our agent's most complex thought process yet, and the LangSmith trace is essential for understanding what actually happened. The agent's reasoning is not a simple, clean line; it's an iterative, multi-step process. This is the "glass box" in action.

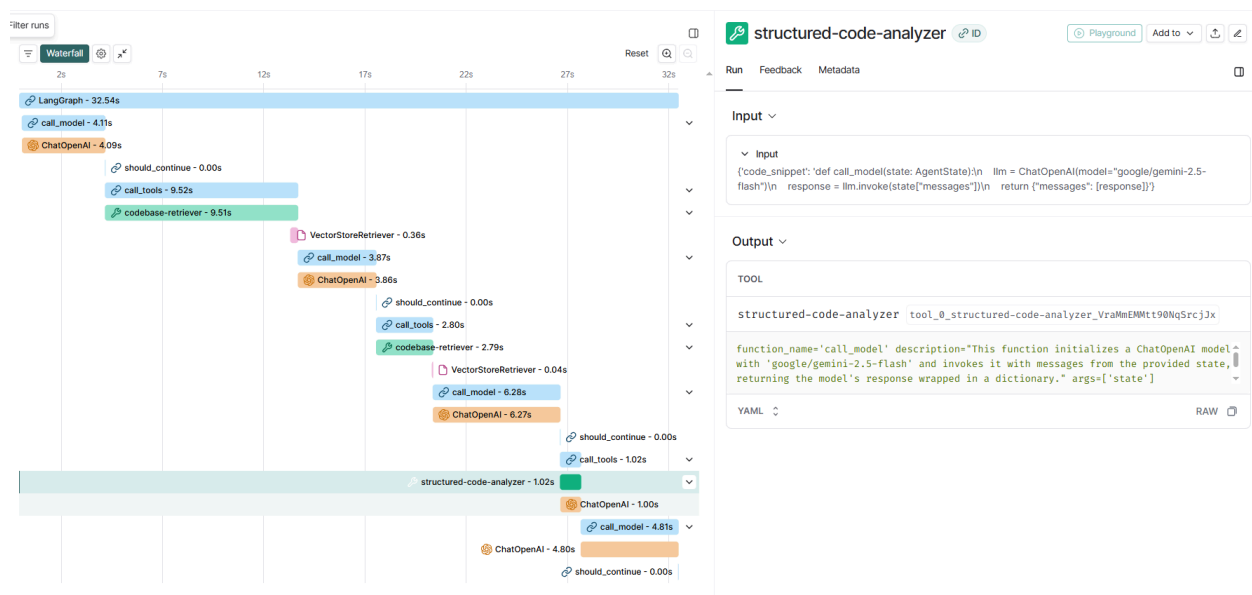


Figure 6-2: Visualizing the Router Agent's Thought Process.

This LangSmith trace is the definitive record of our agent's multi-step reasoning. The waterfall view (left) shows the full, sometimes iterative, chain of thought, while the detail view (right) allows us to inspect the Input and Output of any step, such as the final, successful call to the structured-code-analyzer.

Let's deconstruct the ground truth of this trace:

1. **First `call_model` Step:** The agent receives your query. It correctly reasons that to perform a "structured analysis," it first needs the code. It decides to call the `codebase-retriever` tool. So far, so good.
2. **Iterative Retrieval:** Here is a fascinating and realistic insight. After the first retrieval, the agent's next `call_model` step is *not* to call the analyzer. Instead, it decides to call the `codebase-retriever` **a second time!** This is the "Slacker" or "Confused Intern" in action. It wasn't fully satisfied with its first search, so it tried again. This is a perfect example of why observability is critical—without this trace, this iterative, inefficient behavior would be invisible.
3. **The Correct Routing Decision:** Finally, after its second retrieval, the third `call_model` step has enough context. It now correctly identifies that the `structured-code-analyzer` is the right tool for the job and calls it, passing the retrieved code snippet as the *Input* (visible in the right-hand panel).
4. **The Ground Truth Output:** The `structured-code-analyzer` tool executes. As you can see in the *Output* panel, it successfully produces our structured data: the raw, serialized string `function_name='call_model' . . .`. This is the ground truth that our resilient parser is designed to capture.
5. **Final Synthesis Step:** The graph does not end there. The agent performs one final `call_model` step. Its job here is to take the structured tool output and formulate a final, conversational response for the user. This is the "helpful" step that turns our structured data back into a less-useful string, and it's exactly why our resilient parser, which ignores this final message, is a production-grade necessity.

You have just built and observed a real-world agentic workflow. It's not always a straight line, but it is a logical, observable, and ultimately successful process.

Summary & Key Takeaways

We have now erected our Third Pillar: **Reliability**. By defining a strict data schema with Pydantic and binding it to our LLM, we have tamed the model's unstructured nature and forced it to communicate in the clean, predictable language of data structures. The days of brittle string parsing are over.

Mental Models You've Gained:

- **The Unreliability of Strings:** You now viscerally understand why parsing natural language is a losing battle for application development.
- **Schema as a Contract:** You see Pydantic models as an unbreakable contract that defines what "correct" data looks like.
- **The "Tool-within-a-Tool" Pattern:** You can build sophisticated agents by creating specialized, intelligent tools that encapsulate their own internal LLM-powered logic.
- **Agent as a Router:** Your main agent is evolving from a simple doer into a smart router, deciding which specialized tool is best for the current task.

What's Next

Our agent is now observable, knowledgeable, and reliable. It is a powerful engineering tool. But we have been operating in a safe, trusted environment. What happens when we expose this powerful tool to the unpredictable, and sometimes malicious, outside world?

In Lesson 7, we will put on our security hats. We will harden our agent against real-world threats like prompt injection, building the essential guardrails required for any production system.